Mensajería con Apache Kafka en Pedidos360

Conceptos, arquitectura de streaming, integración de microservicios, administración de tópicos y observabilidad con Kafka UI, Prometheus y Grafana.

DSY1107 · Desarrollo Cloud Native I · Evaluación Parcial N.º 6

Octubre de 2026 · Documento técnico y académico basado en el código fuente suministrado.

Elementos de la solución	Configuración identificada
Kafka	3 brokers · ZooKeeper (3 nodos)
Tópicos de negocio	pedidos.eventos · pagos.eventos · envios.eventos
Distribución	3 particiones y 3 réplicas por tópico
Integraciones	Spring Kafka y RabbitMQ conservado
Observabilidad	Kafka UI · Kafka Exporter · Prometheus · Grafana

Alcance: la documentación describe el código incluido en el ZIP. Los resultados operativos deben verificarse al iniciar Docker; el informe no sustituye las pruebas de ejecución.

Contenido

1. Introducción y propósito
2. Fundamentos de Kafka y streaming
3. Arquitectura del clúster y disponibilidad
4. Tópicos, particiones, réplicas y retención
5. Flujo Kafka y RabbitMQ en Pedidos360
6. Código: productores, consumidores y contratos
7. Automatización mediante Docker Compose
8. Kafka UI, Prometheus y Grafana
9. Pruebas funcionales, carga y tolerancia a fallos
10. Evaluación de diseño y mejoras
11. Glosario y referencias

1. Introducción y propósito

Pedidos360 es una aplicación de compras de videojuegos organizada en un frontend Angular, API Gateway y microservicios Spring Boot con bases PostgreSQL separadas. La arquitectura integra comunicación síncrona para las validaciones de la compra y mensajería asíncrona después de registrarla.

La incorporación de Apache Kafka no reemplaza RabbitMQ. Kafka conserva los hechos de negocio en registros particionados que pueden consultar distintos grupos de consumidores; RabbitMQ continúa resolviendo tareas operativas como registrar el envío, enviar correos, confirmar notificaciones, reintentar fallos y redirigir mensajes a colas de error.

Objetivos

• Explicar tres nodos ZooKeeper, tres brokers y su interacción en Docker Compose.

• Justificar el uso de tres tópicos, cada uno con tres particiones y factor de replicación tres.

• Mostrar productores, consumidores y eventos reales, diferenciando la publicación de la reacción de negocio.

• Interpretar métricas de offsets, consumer lag, disponibilidad de brokers y sincronía de réplicas.

• Reconocer los límites de confiabilidad y proponer pruebas reproducibles.

2. Fundamentos de Kafka y streaming

Concepto	Definición y aplicación
Broker	Servidor Kafka que conserva particiones, atiende productores y consumidores y replica datos.
Topic (tópico)	Registro lógico de eventos; Pedidos360 dispone de tres tópicos de negocio.
Partition	Secuencia ordenada de registros; el orden solo está garantizado dentro de cada partición.
Producer	Componente que agrega eventos al log; utiliza KafkaTemplate en Spring Kafka.
Consumer group	Conjunto de consumidores que comparte particiones de un tópico.
Offset	Posición de un registro dentro de una partición; no es un identificador global.
Retention	Política que conserva registros durante un periodo o tamaño configurado, independientemente de su lectura.

Analogía

Kafka se asemeja a un archivo de movimientos: cada registro queda ordenado por página (partición), varios lectores pueden recorrerlo a su ritmo y el archivo se mantiene según una política de retención. RabbitMQ se parece más a una bandeja de tareas: se confirma el trabajo y el mensaje se retira de la cola.

3. Arquitectura del clúster y disponibilidad

El Docker Compose entregado define zookeeper-1, zookeeper-2 y zookeeper-3, junto con kafka-1, kafka-2 y kafka-3 sobre la red interna. Los brokers se anuncian por nombre de servicio y puerto 9092. La configuración Kafka 7.7.1 utilizada corresponde al despliegue basado en ZooKeeper, no a KRaft.

Capa	Responsabilidad
ZooKeeper (tres nodos)	Coordina metadatos y elección del controlador en el modo ZooKeeper. Tres nodos permiten mantener quórum si falla uno, siempre que los otros dos se comuniquen.
Kafka (tres brokers)	Distribuye líderes y réplicas de particiones; los productores usan la lista de bootstrap servers.
Kafka UI	Visualiza brokers, tópicos, particiones, offsets y mensajes.
Kafka Exporter	Recopila información Kafka y la expone como métricas Prometheus.

Los brokers configuran replication.factor=3 para los tópicos creados por kafka-init, min.insync.replicas=2 y deshabilitan la creación automática implícita de tópicos. La tolerancia real a fallos requiere comprobar ISR, líderes, productores y consumidores durante las pruebas.

4. Tópicos, particiones, réplicas y retención

Tópico	Función identificada	Configuración
pedidos.eventos	Registra PEDIDO_COMPLETADO desde msvc-pago	3 particiones, RF 3
pagos.eventos	Registra el mismo sobre de compra emitido por msvc-pago	3 particiones, RF 3
envios.eventos	Publica ENVIO_REGISTRADO tras guardar un envío en msvc-envio	3 particiones, RF 3

Con clave pedidoId, los registros de un pedido se dirigen de forma estable a una partición dentro de cada tópico; no se garantiza orden transversal entre los distintos tópicos. El factor de replicación 3 genera tres copias de cada partición; no multiplica el número lógico de eventos.

La condición ISR incluye las réplicas sincronizadas; min.insync.replicas=2 es relevante para las escrituras con acks=all. Para afirmar garantías específicas es necesario comprobar los valores efectivos del productor. No se debe inferir tolerancia completa a partir de que aparezcan tres brokers en Kafka UI.

Retención

El script de inicialización configura particiones y réplicas, pero no fija una política personalizada de retención para estos tópicos. Por ello aplica la configuración heredada del broker. Para una evaluación se recomienda mostrar retention.ms/retention.bytes efectivos y justificar su valor antes de modificarlos.

5. Flujo Kafka y RabbitMQ en Pedidos360

Etapa	Implementación en el proyecto
1. Compra	Angular llama al API Gateway; msvc-pago confirma el pedido tras verificar la operación.
2. Streaming	CompraKafkaPublisher envía un sobre JSON a pedidos.eventos y pagos.eventos.
3. Colas operativas	La publicación RabbitMQ pago.completado activa el registro del envío y la notificación por correo de forma independiente.
4. Registro de envío	msvc-envio persiste un Envio y EnvioKafkaPublisher publica ENVIO_REGISTRADO.
5. Analítica	Los consumidores Kafka de msvc-envio y msvc-catalogo registran los mensajes recibidos para seguimiento.
6. Observabilidad	Kafka UI muestra los mensajes; exporter y Prometheus alimentan el dashboard de Grafana.

El canal Kafka sirve para conservar y analizar hechos. El servicio de envío no debe duplicar su registro solo porque consuma el tópico de pagos: el procesamiento operativo existente se mantiene en RabbitMQ.

6. Código: productores, consumidores y contratos

6.1 msvc-pago: CompraKafkaPublisher

Implementado en msvc-pago/.../kafka/CompraKafkaPublisher.java. Inyecta KafkaTemplate<String,String> y ObjectMapper, serializa el sobre y publica asíncronamente en dos tópicos. Incluye eventId, eventType PEDIDO_COMPLETADO, schemaVersion, occurredAt, pedidoId, numeroPedido, total y cantidadItems.

```
kafka.send(topic, id.toString(), data)
    .whenComplete((result, error) -> { ... });
```

6.2 msvc-envio: EnvioKafkaPublisher

Implementado en msvc-envio/.../kafka/EnvioKafkaPublisher.java. Tras registrar el envío, publica eventType ENVIO_REGISTRADO con envioId, pedidoId, numeroPedido y estadoNuevo. El productor registra partición y offset o registra el fallo asíncrono en los logs.

6.3 Consumidores Kafka

En msvc-envio se declara un KafkaListener del tópico pagos.eventos y el grupo envio-analitica con concurrency=3; en msvc-catalogo existe otro consumidor para analítica. Los listeners identificados escriben información en logs; esto no equivale a afirmar que realizan el registro operativo de envío, que se conserva en RabbitMQ.

6.4 Contratos: evento frente a comando

Los mensajes expresan hechos en pasado, no órdenes de ejecución. El campo schemaVersion permite evolucionar los contratos; eventId facilita deduplicación futura, aunque el productor actual no implementa por sí solo una tabla inbox.

7. Automatización mediante Docker Compose

La única entrada de despliegue se encuentra en la raíz del proyecto. El script pedidos-backend/kafka/init-topics.sh espera la disponibilidad de Kafka y crea idempotentemente los tres tópicos con --if-not-exists. El servicio kafka-init termina correctamente con código 0 una vez creado el esquema. Los microservicios que dependen de su inicialización esperan service_completed_successfully.

```
docker compose up -d --build
docker compose ps -a
docker compose logs kafka-init --tail=50
```

Kafka Exporter utiliza restart: unless-stopped para recuperarse después de errores de arranque. Prometheus y Grafana también conservan una política de reinicio. Los volúmenes con nombre mantienen bases de datos y datos Kafka al ejecutar docker compose down sin -v.

8. Kafka UI, Prometheus y Grafana

Interfaz	Puerto local	Qué demostrar
Kafka UI	8088	Brokers, tópicos, mensajes, offsets y consumer groups
Prometheus	9090	Targets UP/DOWN; consultas PromQL
Grafana	3000	Dashboard Pedidos360 | Apache Kafka
RabbitMQ Management	15672	Exchanges, colas, reintentos y DLQ

Métricas de interés

PromQL	Interpretación
kafka_brokers	Cantidad de brokers visibles desde el exporter
kafka_topic_partitions	Número de particiones por tópico
kafka_topic_partition_current_offset	Desplazamiento actual por partición (útil para observar nuevos registros)
kafka_consumergroup_lag	Diferencia entre producción y posición confirmada de un grupo; debe investigarse si aparece sin series, negativa o crece sostenidamente.

En Prometheus, el job kafka-exporter debe encontrarse UP, junto con los targets Spring de msvc-pago, msvc-envio y msvc-catalogo. Grafana utiliza como origen Prometheus. Los paneles de tendencia representan métricas, no sustituyen la inspección del JSON real en Kafka UI.

9. Pruebas funcionales, carga y tolerancia a fallos

Prueba	Pasos / evidencia esperada
Arranque nuevo	docker compose up -d --build; verificar kafka-init Exited (0), tópicos y tres brokers.
Compra completa	Realizar compra Angular; revisar JSON en pedidos.eventos y pagos.eventos; revisar el registro en RabbitMQ y el evento en envios.eventos.
Consumidores	En Kafka UI identificar grupos envio-analitica y catalogo-analitica; examinar offsets/lag antes y después de compras.
Fallo de un broker	Detener uno de los brokers; observar líder, ISR y recuperación; restaurarlo antes de otras pruebas.
Servicio consumidor detenido	Detener msvc-envio en entorno local de pruebas; generar tráfico controlado; reanudar y observar procesamiento pendiente de RabbitMQ.
Exporter detenido	Detener kafka-exporter: Prometheus marca DOWN; al reanudar, deben volver las métricas.

Comandos reproducibles

```
docker compose exec kafka-1 kafka-topics --bootstrap-server kafka-1:9092 --list
docker compose exec kafka-1 kafka-topics --bootstrap-server kafka-1:9092 \
  --describe --topic envios.eventos
docker compose logs -f msvc-pago msvc-envio
```

Las pruebas de caída deben realizarse en una copia local controlada; no borrar volúmenes ni reinicializar bases de datos que contengan compras reales. Registrar capturas antes, durante y después del incidente.

10. Evaluación de diseño y mejoras

Fortaleza observada	Límite o mejora
Distribución y réplica por tópicos	La replicación protege disponibilidad del log, pero deben comprobarse acks e ISR ante fallos.
Grupos de consumidores para analítica	La carga depende del número de particiones y del trabajo efectivo de los listeners.
Eventos con identificadores	Agregar contratos tipados versionados, validación y estrategia de compatibilidad de esquemas.
Publicaciones asíncronas	No hay Atomicidad BD-Kafka; incorporar Transactional Outbox y procesamiento idempotente.
Monitoreo preparado	Agregar alertas por exporter caído, bajo ISR, lag y errores de publicación.
RabbitMQ y Kafka coexistentes	Documentar límites de responsabilidad para evitar que un evento duplique efectos de negocio.

11. Glosario y referencias

Término	Significado
ISR	In-Sync Replicas: conjunto de réplicas al día con el líder.
Leader	Réplica que atiende escrituras/lecturas de la partición.
Consumer lag	Retraso de un grupo respecto al último registro del tópico.
Bootstrap server	Dirección inicial para descubrir el clúster y sus brokers.
Offset	Posición de un evento en una partición.
Replay	Relectura de eventos retenidos desde offsets anteriores.
Transactional Outbox	Almacena dato de negocio y evento pendiente en una misma transacción local.

Referencias documentales: código fuente Pedidos360 (Docker Compose, kafka/init-topics.sh, productores/consumidores y observabilidad); informe académico «Mensajería con colas y RabbitMQ en Pedidos360» facilitado como antecedente metodológico; documentación oficial de Apache Kafka, Spring for Apache Kafka, Prometheus y Grafana.

Este informe describe y analiza los archivos entregados. Las capturas e indicadores operativos presentados en clase deben obtenerse ejecutando el entorno local y guardando las evidencias correspondientes.